Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la lucrarea de laborator nr. 8

Disciplina: Tehnici de programare

Tema: Colecții în Python
Liste, tupluri, mulțimi și dicționare

A efectuat: Moraru Elena, grupa SI-264

A verificat: asistent universitar Reițman Patricia

Chișinău 2026

Lucrarea A. Enumerarea și gruparea datelor

Scopul lucrării este utilizarea listelor, tuplurilor, mulțimilor și dicționarelor pentru prelucrarea fișierelor SUID, a regulilor sudo și a sarcinilor cron. Programele citesc datele furnizate și rulează local, cu Python 3.14.7.

A1. Citirea liniilor într-o listă

Funcția elimină liniile goale și comentariile. Lista păstrează ordinea și dublurile: fișierul suid.txt conține 14 linii, inclusiv repetarea lui /usr/bin/passwd.

```
11  def citeste_linii(cale):
12      """Pastreaza ordinea si dublurile; elimina golurile si comentariile."""
13      with open(cale, encoding="utf-8") as fisier:
14          return [ln.strip() for ln in fisier
15                  if ln.strip() and not ln.lstrip().startswith("#")]
```

Fig. 1.1. Funcția citeste_linii din enum.py.

```
linii SUID: 14
```

Fig. 1.2. Numărul liniilor citite din suid.txt.

A2. Extragerea numelor unice

Numele se extrage după ultimul caracter „/”. Mulțimea creată prin set comprehension elimină repetarea și conține 13 nume distincte.

```
18  def nume(cale):
19      """Extrage numele unei cai Linux, inclusiv cand rulam pe Windows."""
20      return cale.rsplit("/", 1)[-1]
```

Fig. 1.3. Extragerea numelui unui fișier.

```
3  start = 'arsenal/privesc/start'
4  c = citeste_linii(start + '/suid.txt')
5  u = {nume(x) for x in c}
6  print('SUID distincte:', len(u))
```

Fig. 1.4. Construirea mulțimii în exemplul executabil run_A2.py.

```
SUID distincte: 13
```

Fig. 1.5. Numărul numelor SUID distincte.

A3. Lista constatărilor SUID

Fiecare cale unică este reprezentată printr-un dicționar cu cheile cale, nume și risc. Căile sunt sortate. Numele din mulțimea GTFO primesc eticheta „mare”.

```
GTFO = {"find", "vim.basic", "vim", "nmap", "cp", "bash",
        "less", "awk", "python3", "perl"}
```

```
def suid_constatari(cale):
    """O fisa pentru fiecare cale unica, in ordine determinista."""
    return [{"cale": c, "nume": nume(c),
             "risc": "mare" if nume(c) in GTFO else "normal"}
            for c in sorted(set(citeste_linii(cale)))]
```

Fig. 1.6. Mulțimea GTFO și construirea listei de dicționare în enum.py.

```
13 SUID, 4 cu risc mare
['cp', 'find', 'nmap', 'vim.basic']
```

Fig. 1.7. Cele patru nume marcate cu risc mare.

A4. Regulile sudo, sarcinile cron și raportul comun

Regula sudo se despachetează în user și drept. Pentru cron se citesc cinci câmpuri de timp, apoi utilizatorul și comanda. split(maxsplit=6) păstrează comanda completă. Funcția enumereaza reunește cele trei liste într-un dicționar.

```
def sudo_constatari(cale):
    """Despacheteaza fiecare regula in utilizator si drept."""
    out = []
    for ln in citeste_linii(cale):
        user, drept = ln.split(" : ", 1)
        out.append({"user": user.strip(), "drept": drept.strip(),
                    "risc": "mare" if "NOPASSWD" in drept else "normal"})
    return out


def cron_constatari(cale):
    """Formatul furnizat: cinci campuri de timp, user, comanda."""
    out = []
    for ln in citeste_linii(cale):
        campuri = ln.split(maxsplit=6)
        if len(campuri) != 7:
            raise ValueError(f"Linie cron invalida: {ln}")
        out.append({"user": campuri[5], "comanda": campuri[6]})
    return out


def enumereaza(d):
    """Grupeaza constatarile pe sursa intr-un dictionar de liste."""
    d = Path(d)
    return {"suid": suid_constatari(d / "suid.txt"),
            "sudo": sudo_constatari(d / "sudo.txt"),
            "cron": cron_constatari(d / "crontab.txt")}
```

Fig. 1.8. Parsarea regulilor sudo și cron și gruparea pe categorii.

```
ana mare
ana normal
www-data mare
root cd / && run-parts --report /etc/cron.hourly
root /opt/backup/run.sh
www-data /home/www-data/clean.sh
```

Fig. 1.9. Regulile sudo și comenzile cron citite din fișiere.

A5. Raportul JSON și rezumatul pe categorii

Argumentul capturi indică directorul datelor. Cu --json se afișează numai JSON; fără acest comutator se afișează rezumatul și se salvează enum.json. Opțiunea --output permite alegerea fișierului de ieșire.

```
81  def main():
82      p = argparse.ArgumentParser(description=__doc__)
83      p.add_argument("capturi", type=Path)
84      p.add_argument("--json", action="store_true")
85      p.add_argument("--risc", choices=["mare", "normal"])
86      p.add_argument("--output", type=Path, default=Path("enum.json"))
87      a = p.parse_args()
88      raport = enumereaza(a.capturi)
89      if a.risc:
90          raport = filtreaza_risc(raport, a.risc)
91      if a.json:
92          print(json.dumps(raport, indent=2, ensure_ascii=False))
93          return
94      for categorie, lista in raport.items():
95          mari = [c for c in lista if c.get("risc") == "mare"]
96          print(f"{categorie}: {len(lista)} gasite, {len(mari)} cu risc mare")
97      print("utilizatori implicati:", ", ".join(sorted(utilizatori(raport))))
98      with a.output.open("w", encoding="utf-8") as f:
99          json.dump(raport, f, indent=2, ensure_ascii=False)
100         f.write("\n")
101     print(f"Raport salvat in {a.output.as_posix()}")
102
103
104 if __name__ == "__main__":
105     main()
```

Fig. 1.10. Funcția main și salvarea raportului JSON în enum.py.

Utilizatorii implicați se obțin prin reuniunea mulțimilor din sudo și cron.

```
59  def utilizatori(raport):
60      return ({c["user"] for c in raport["sudo"]}
61              | {c["user"] for c in raport["cron"]})
```

Fig. 1.11. Reuniunea utilizatorilor din cele două categorii.

```
suid: 13 gasite, 4 cu risc mare
sudo: 3 gasite, 2 cu risc mare
cron: 3 gasite, 0 cu risc mare
utilizatori implicati: ana, root, www-data
Raport salvat in enum.json
```

Fig. 1.12. Rezumatul afișat de program și confirmarea salvării.

Comanda de rulare: `python -m arsenal.privesc.enum arsenal/privesc/start`. Raportul conține 13 constatări SUID, 3 reguli sudo și 3 sarcini cron. Utilizatorii sunt ana, root și www-data.

Lucrarea B. Comparația cu sistemul de referință

B1. Mulțimea de referință și mulțimea curentă

Funcția cai transformă liniile citite într-o mulțime. Fișierul baza_suid.txt conține 9 căi distincte, iar suid.txt conține 13.

```
 9  def cai(fisier):
10      return set(citeste_linii(fisier))
```

```
26      baza = cai(a.capturi / "baza_suid.txt")
27      acum = cai(a.capturi / "suid.txt")
28      rezultat = compara(baza, acum)
```

Fig. 2.1. Citirea celor două mulțimi în diferenta.py.

```
baza: 9  acum: 13
```

Fig. 2.2. Dimensiunile mulțimilor de referință și curentă.

B2. Identificarea căilor noi

Diferența acum - baza păstrează căile prezente în captura curentă și absente din imaginea de referință. Sortarea asigură aceeași ordine la fiecare rulare.

```
13  def compara(baza, acum):
14      noi = acum - baza
15      risc = {nume(c): ("mare" if nume(c) in GTFO else "normal")
16              for c in sorted(noi)}
17      periculoase = [c for c in sorted(noi) if nume(c) in GTFO]
18      return {"noi": sorted(noi), "risc": risc, "periculoase": periculoase}
```

Fig. 2.3. Calcularea diferenței și pregătirea clasificării în funcția compara.

```
32      print("baza:", len(baza), " acum:", len(acum))
33      print(len(rezultat["noi"]), "SUID noi fata de sistemul curat")
34      for c in rezultat["noi"]:
35          print(" ", c)
```

Fig. 2.4. Afișarea numărului și a căilor noi.

```
4 SUID noi fata de sistemul curat
  /bin/cp
  /usr/bin/find
  /usr/bin/nmap
  /usr/bin/vim.basic
```

Fig. 2.5. Cele patru căi apărute față de sistemul de referință.

B3. Clasificarea și filtrarea rezultatelor

Dicționarul risc asociază fiecărui nume nou eticheta „mare” sau „normal”. Lista periculoase păstrează numai căile al căror nume se găsește în GTFO. Funcția compara este prezentată în Fig. 2.3. Toate cele patru căi noi din datele furnizate sunt marcate cu risc mare.

```
13  def compara(baza, acum):
14      noi = acum - baza
15      risc = {nume(c): ("mare" if nume(c) in GTFO else "normal")
16              for c in sorted(noi)}
17      periculoase = [c for c in sorted(noi) if nume(c) in GTFO]
18      return {"noi": sorted(noi), "risc": risc, "periculoase": periculoase}
```

Fig. 2.6. Dict comprehension și list comprehension pentru clasificare.

```
periculoase NOI:
/bin/cp
/usr/bin/find
/usr/bin/nmap
/usr/bin/vim.basic
```

Fig. 2.7. Căile noi care trec filtrul de risc.

B4. Testul de regresie

Testul verifică atât operația de diferență cerută în fișă, cât și rezultatul funcției compara. Astfel, inversarea argumentelor ar produce un test eșuat.

```
5   def test_doar_ce_e_nou():
6       acum = {"a", "b", "c"}
7       baza = {"a", "b"}
8       assert acum - baza == {"c"}
9       assert compara(baza, acum)["noi"] == ["c"]
```

Fig. 2.8. Testul test_doar_ce_e_nou din test_diferenta.py.

```
.                                                                        [100%]
1 passed in 0.05s
```

Fig. 2.9. Rezultatul executării testului de regresie cu pytest.

Au fost verificate suplimentar datele furnizate, eliminarea golurilor și comentariilor, direcția diferenței, o cale nouă cu risc normal, ieșirea JSON și salvarea acesteia.

```
......                                                                   [100%]
6 passed in 1.06s
```

Fig. 2.10. Rezultatul suitei complete: toate cele șase teste trec.

Extensii opționale și concluzii

Filtrarea și gruparea după risc

Comutatorul --risc mare filtrează constatările din fiecare categorie. Gruparea cu setdefault adună constatările în liste după risc. Sarcinile cron nu au câmpul risc și sunt grupate ca „neclasificat”.

```
64  def filtreaza_risc(raport, risc):
65      """Extindere optionala: filtrare in toate categoriile."""
66      return {cat: [c for c in lista if c.get("risc") == risc]
67              for cat, lista in raport.items()}
68
69
70  def grupeaza_risc(raport):
71      """Extindere optionala: campul absent devine neclasificat."""
72      grupuri = {}
73      for categorie, lista in raport.items():
74          for c in lista:
75              eticheta = c.get("nume", c.get("user", ""))
76              grupuri.setdefault(c.get("risc", "neclasificat"), []).append(
77                  f"{categorie}: {eticheta}")
78      return grupuri
```

Fig. 3.1. Funcțiile filtreaza_risc și grupeaza_risc din enum.py.

```
{'suid': 4, 'sudo': 2, 'cron': 0}
{'mare': 6, 'normal': 10, 'neclasificat': 3}
```

Fig. 3.2. Numărul constatărilor filtrate și grupate după risc.

Concluzii

Lista păstrează liniile și ordinea lor, tuplul permite despachetarea regulilor sudo, mulțimea elimină dublurile și calculează diferența, iar dicționarul descrie și grupează constatările. Pe datele analizate au fost identificate patru căi SUID noi: /bin/cp, /usr/bin/find, /usr/bin/nmap și /usr/bin/vim.basic.

Eticheta de risc este regula de clasificare folosită în exercițiu. Ea indică elementele care trebuie verificate, fără a demonstra obținerea accesului root. Lipsa etichetei de risc pentru cron nu reprezintă o garanție de siguranță.

Rulare

Programele folosesc biblioteca standard Python. Pentru teste este necesar pytest. Comenzile se execută din rădăcina proiectului:

```
python -m arsenal.privesc.enum arsenal/privesc/start
python -m arsenal.privesc.enum arsenal/privesc/start --json
python -m arsenal.privesc.diferenta arsenal/privesc/start
python -m pytest -q arsenal/privesc/test_diferenta.py tests
```